

understanding

<https://f1tenth-coursekit.readthedocs.io/en/latest/assignments/labs/lab2.html>

Virtual LiDAR:

- Scans approximately a 269° field of view, not a full circle.
- Divides that area into 1080 directions, or rays.
- Measures the distance to the nearest obstacle in each direction.
- Publishes those 1080 distances in:

```
scan.ranges
```

Conceptually:

```
ray 0      -> distance in direction 0
ray 1      -> distance in direction 1
...
ray 1079   -> distance in direction 1079
```

The LiDAR does not send 1080 physical points. It sends 1080 distance measurements along 1080 rays. In your simulator, each ray is separated by approximately 0.249° .

Basic Algorithm

so i choose the ray with max distance assuming its the safe direction then compute that angle and then compute steering

1. Find the ray with the greatest distance:

```
best_index = np.argmax(scan.ranges)
```

2. Convert its index into an angle:

```
best_angle = scan.angle_min + best_index * scan.angle_increment
```

3. Use that angle as the steering angle:

```
drive.drive.steering_angle = best_angle
```

4. Publish the drive command.

However, the farthest ray is not always safe by itself. A single far ray may be a noisy reading or may point beside an obstacle. Better agents usually:

- Ignore rays behind the car
- Remove invalid readings
- Create a safety bubble around nearby obstacles
- Choose the best point inside a wide gap
- Reduce speed for large steering angles
- Smooth sudden steering changes

So the basic algorithm is:

```
LiDAR distances
-> find safe open region
-> choose target ray
-> convert ray index to angle
-> publish steering and speed
```

versions notes

Drive_agent.py

Very basic, just a drive command publisher, to check subscriber publisher working and topics

Safety_braking_agent.py

Simple Time to collision braking system, calculates time to collide rather than just slamming brakes after detecting an obstacle.

$$iTTC = \frac{r}{\{-\dot{r}\}_+}$$

PID Wall Follower

Just follow a side of the track, keeping a const distance from the wall, uses 2 beams to find the orientation of the car and predicts just a bit ahead and then regular pid to maintain the position

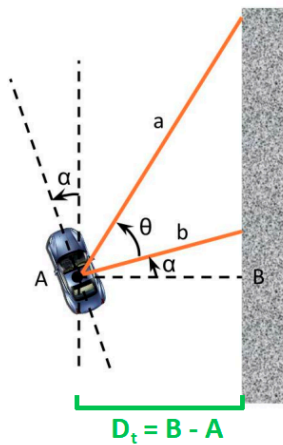


Figure 1: Distance and orientation of the car relative to the wall

Using the two distances a and b from the laser scan, the angle θ between the laser scans, and some trigonometry, we can express α as

$$\alpha = \tan^{-1} \left(\frac{a \cos(\theta) - b}{a \sin(\theta)} \right)$$

We can then express D_t as

$$D_t = b \cos(\alpha)$$

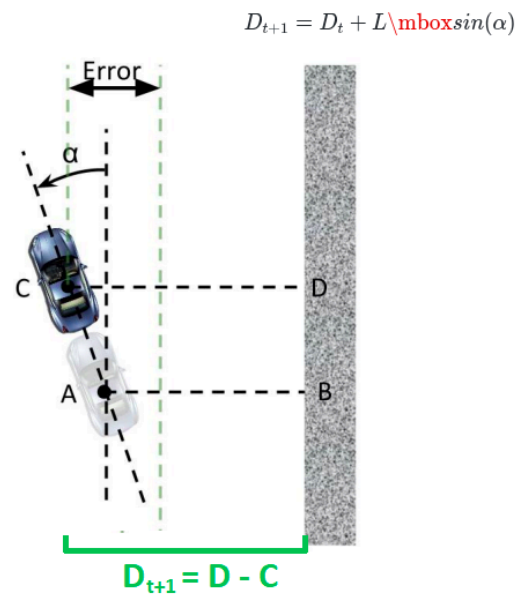
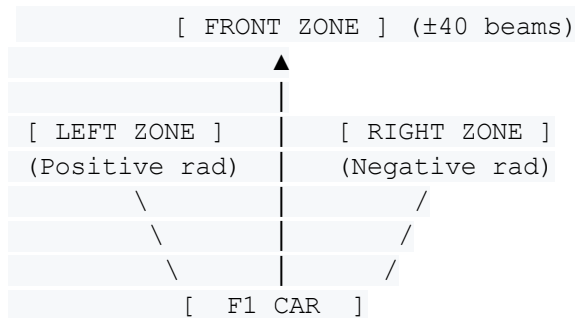


Figure 2: Finding the future distance from the car to the wall

Simple_lidar_avoider.py

Core Goal: Move forward, and if something blocks the front, turn hard toward the side with more open space.



How the logic works in 3 simple steps:

Slices the LiDAR into 3 Zones:

- **Front:** Narrow cone directly ahead (± 40 ± 40 beams around center).
- **Left:** Sector of beams to the left of center.
- **Right:** Sector of beams to the right of center.

(LiDAR angles in ROS 2: Counter-Clockwise $\rightarrow \rightarrow$ Negative = Right, Positive = Left).
Measures Clearances:

front_distance: Finds the closest obstacle directly in front.

left_clearance & right_clearance: Finds the maximum open space on either side.

Makes a Steering Decision (If-Else):

- **Path Clear** ($\text{front_distance} > 1.0 \text{ m}$): Drive straight at 2.0 m/s.
- **Obstacle Detected** ($\text{front_distance} \leq 1.0 \text{ m}$):

If $\text{right_clearance} > \text{left_clearance} \rightarrow \rightarrow$ Steer hard right ($- 0.55 \text{ rad}$ -0.55 rad) at 1.4 m/s 1.4 m/s. Otherwise $\rightarrow \rightarrow$ Steer hard left ($+ 0.55 \text{ rad}$ $+0.55 \text{ rad}$) at 1.4 m/s 1.4 m/s.

- **Safety Stop:** If any obstacle anywhere gets closer than 0.25 m 0.25 m, emergency stop (speed = 0.0).

Follow the Gap

Reactive agent, reacts to obstacles, if a obstacle detected makes a safety bubble around it, Looks at a band of lidar rays, directions with obstacle and the safety bubble are made 0, and the widest non zero band of integers is considered, and the safest direction among these is found and followed

Pure Pursuit

Follows a pre recorded optimised race path, with highly optimised velocity control, No obstacle detection or anything, just repeating the predefined path with max speed

Dynamic Overtaker

Pure Pursuit but with obstacle avoidance too, follows the pursuit track but steers to left or right to avoid and overtake the obstacle

quick commands


```
//STOPP
ros2 topic pub --once /drive \
  ackermann_msgs/msg/AckermannDriveStamped \
  "{drive: {speed: 0.0, steering_angle: 0.0}}"
```

TAB 1

```
cd /home/ms/sim_ws

sudo rocker --x11 \
  --env LIBGL_ALWAYS_SOFTWARE=1 \
  --env GALLIUM_DRIVER=llvmpipe \
  --volume "$PWD:/sim_ws" \
  -- f1tenth_gym_ros

source /opt/ros/foxy/setup.bash
source /sim_ws/install/local_setup.bash

ros2 launch f1tenth_gym_ros gym_bridge_launch.py
```

TAB 2

```
docker ps

docker exec -it CONTAINER_NAME bash

source /opt/ros/foxy/setup.bash

source /sim_ws/install/local_setup.bash

python3
/sim_ws/src/f1tenth_gym_ros/references/giu_gap_follower/gap_follower/steering_speed_control.py --ros-args -p drive_topic:=/drive -p control_selector_topic:=/control_selector
```

TAB 3

```
docker exec -it CONTAINER_NAME bash
source /opt/ros/foxy/setup.bash

ros2 topic pub --rate 2 /control_selector std_msgs/msg/String "{data: gap_following}"
```

```
cd /sim_ws
```

```
colcon build --packages-select ms_agents_pkg --symlink-install
```

```
source install/local_setup.bash
```